

```

        if '.' in self.text:
            return False # can't enter a second period

# Allow the key to go through to the base class
result = super().handleEvent(event)
return result

def getValue(self): ❹
    userString = super().getValue()
    try:
        if self.allowFloatingNumber:
            returnValue = float(userString)
        else:
            returnValue = int(userString)
    except ValueError:
        raise ValueError('Entry is not a number, needs to have at least one
digit.')

    return returnValue

```

Listing 10-3: InputNumber only allows the user to enter numeric data.

The `__init__()` method allows for the same parameters as the `InputText` base class, plus a few more ❶. It adds two Booleans: `allowFloatingNumber` to determine if the user should be allowed to enter floating-point numbers and `allowNegativeNumber` to determine if the user can enter a number starting with a minus sign. Both default to `True`, so the default case allows the user to enter a floating-point number and both positive and negative numbers. You could use these to restrict the user to, for example, only entering a positive integer value by setting both to `False`. The `__init__()` method saves the values of these two additional parameters in instance variables, then calls the `__init__()` method of the base class using the call to `super()` ❷.

The significant code is in the `handleEvent()` method ❸, which restricts the allowed keys to a small subset: the numbers zero through nine, the minus sign, a period (decimal point), ENTER, and a few editing keys. When the user presses a key, this method is called and a `KEYDOWN` OR `KEYUP` event is passed in. The code first ensures that the key pressed is in the restricted set. If the user enters a key not in that set (for example, any letter), we return `False` to indicate that nothing important has happened in this widget, and that key is ignored.